

บทที่ 2

ทฤษฎีที่เกี่ยวข้อง

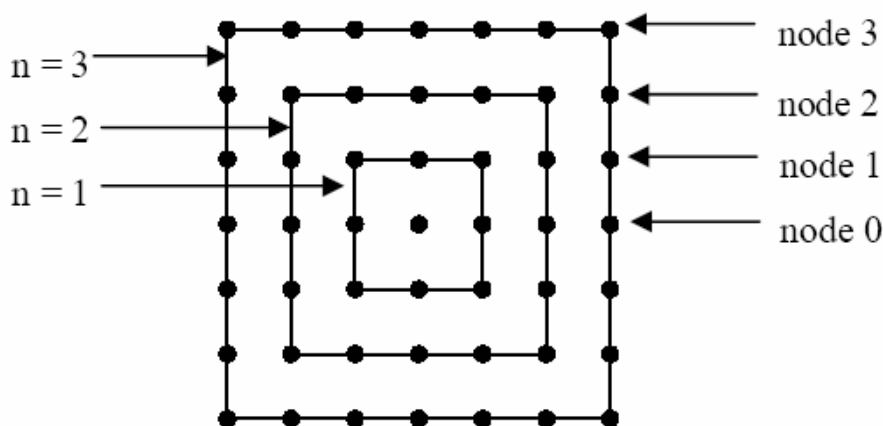
2.1 ทฤษฎีที่ใช้ในการพัฒนาโปรแกรมในส่วนรู้จำตัวอักษร(โครงการเดิมพัฒนาโดย. คำเพ็ด บุญนะดี)

ข้อมูลที่ได้จากการเขียนตัวอักษรภาษาไทยเข้าสู่คอมพิวเตอร์ในรูปเซตของคู่อันดับหรือ พิกัด(x, y) ซึ่งได้จากการ Sampling ตามเวลาของอุปกรณ์อิเล็กทรอนิกส์ ข้อมูลนี้จะถูกนำมาเข้ารหัสที่เรียกว่า การเข้ารหัสลูกโซ่แบบGCC (Generalize Chain Code) รหัสลูกโซ่แบบGCC ประกอบด้วยตัวแปรสองตัว คือ

n : ตัวแปรกำหนดวงจรรหัสที่มีลำดับเป็น n

node : แต่ละวงจรรหัสประกอบไปด้วย node ทั้งหมด M node เมื่อ $M = (8 \times n)$ และ $n = 1, 2, 3, \dots$

บนพื้นฐานที่กำหนดให้วงจรรหัสมีจุดใจกลางเพียงจุดเดียวซึ่งก็คือ จุดพิกัด (x, y) ตำแหน่งที่พิจารณาไปก่อนหน้านี้ทำให้ทุกๆลายมือเขียนสามารถที่จะเข้ารหัสได้



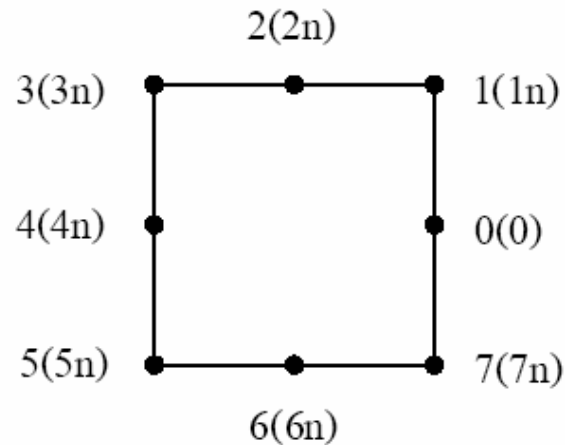
รูปที่ 2.1 แสดงส่วนประกอบของรหัสลูกโซ่แบบ GCC วงจรรหัสของและจำนวน node ที่มีในแต่ละวงจรรหัส

ในการคำนวณหารหัสลูกโซ่แบบ GCC สิ่งหนึ่งที่จะต้องหาก็คือ เวกเตอร์จากจุดกึ่งกลางวงรหัส (คือจุดที่ทำการเข้ารหัสไปก่อนหน้านี้) ไปยัง node ที่อยู่ใกล้ที่สุดที่ติดกับวงรหัส โดยการจะกำหนดว่า node นั้นอยู่วงรหัสที่เท่าใดสามารถคำนวณได้จากผลต่างตามแกน x และ แกน y ของจุด 2 จุดที่มีลำดับติดกันตามสมการที่ (1) โดยผลต่างตามแนวแกนใดมีค่ามากที่สุดก็จะนำมากำหนดลำดับวงรหัส

$$n = |\max(x_2 - x_1)| \text{ หรือ } n = |\max(y_2 - y_1)| \quad (1)$$

จากนั้นทำ Normalized GCC (NGCC) เพื่อลดจำนวนวงรหัสและเพื่อให้มี node เหลือเพียง 8 node หลัก โดย NGCC เป็นรหัส C_i ซึ่งคำนวณได้จาก

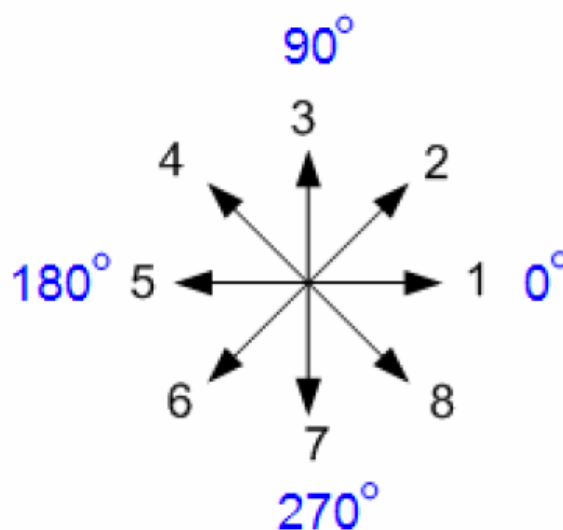
$$C_i = \text{node}/n \quad (2)$$



รูปที่ 2.2 GCC เมื่อทำการ Normalized แล้วจะเหลือเพียง 8 node ต่อวงรหัส

รหัส NGCC C_i จะอธิบายถึงค่า Absolute ของทุกเส้นเชื่อมต่อ (Link) ของ NGCC โดยไม่ขึ้นอยู่กับกำหนัดวงรหัส

ตัวอย่าง ถ้ารหัส NGCC มี เส้นเชื่อมต่อที่มีค่า 0.78 เราจะได้ รหัสสลับไซในระบบเป็น $0.78 + 1 = 1.78$ เพราะรหัสสลับไซที่นำมาใช้ในที่นี้จะเริ่มต้นด้วย 1 จนถึง 8.99 ซึ่งรหัสสลับไซที่มีค่า 1.87 ก็จะได้เส้นเชื่อมต่อยู่ระหว่างสอง node ที่ติดกันคือ node 0 และ node 1 และมุมของเส้นเชื่อมต่อนี้จะมีค่าอยู่ในช่วงของ $0 < \theta < 45$ องศาซึ่งการเข้ารหัส NGCC จะเทียบเท่ากับการทำ quantization มุมดังแสดงในรูปที่ 3



รูปที่ 2.3 รหัสสลับไซแบบ GCC ที่ทำการ quantization แล้ว

ตัวอย่างการหารหัสลูกโซ่แบบ GCC ของตัวอักษรลายมือเขียนโดยสมมุติว่าจุดที่ Sampling ได้จากเขียนนั้นมีจุดดังนี้ P1(344,183) ; P2 (343,186); P3 (342,187); P4 (340,188) ในการคำนวณหารหัสลูกโซ่ระหว่างจุด P1 และ P2 มีดังนี้

ก่อนอื่นจะคำนวณหาวงรหัสดีก่อนว่าอยู่ในวงรหัสนี้เท่าไร ซึ่งคำนวณดังนี้

$$n = |\text{MAX}(P2(X) - P1(X))| = |\text{MAX}(343 - 344)| = 1 \quad \text{หรือ}$$

$$n = |\text{MAX}(P2(Y) - P1(Y))| = |\text{MAX}(186 - 183)| = 3$$

ซึ่งจะเลือกค่าที่มากที่สุดเป็นวงรหัสนี้ก็คือ 3 หลังจากนั้นหารระหว่างสองจุดซึ่งคำนวณหาได้ดังนี้

$$\text{Length} = \sqrt{1^2 + 3^2} = 3.1622$$

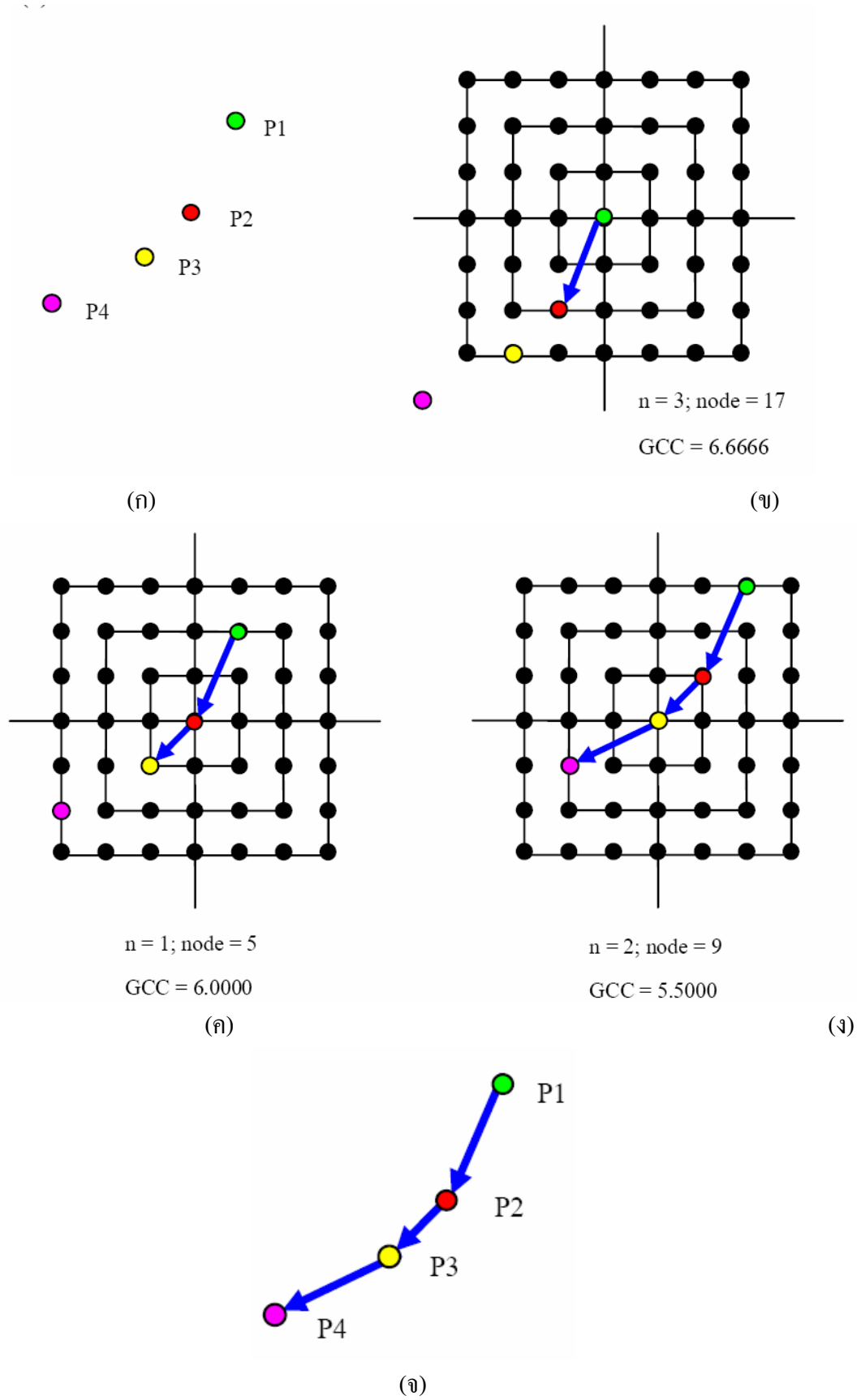
จากนั้นหาว่าจุดที่สนใจนั้นตกอยู่ใน node ที่เท่าใดในวงรหัสนี้ 3 ซึ่งในนี้เป็น node ที่ 17 ดังรูปที่ 4 (ข) แล้วหาว่าเป็นรหัสลูกโซ่ที่เท่าใดซึ่งคำนวณจะได้จาก

$$C_i = \text{node}/n = 17 / 3 = 5.6666$$

แต่เนื่องจากรหัส GCC ที่ใช้นั้นเริ่มต้นจาก 1 เป็นต้นไปดังรหัส

$$\text{GCC} = C_i + 1 = 5.6666 + 1 = 6.6666$$

และจุดต่อไปก็จะทำเช่นเดียวกันนี้ไปเรื่อยไปจนครบทุกจุด



รูปที่ 2.4 แสดงตัวอย่างการเข้ารหัสลูกโซ่แบบ GCC

เนื่องจากการรู้จำตัวอักษรเป็นขบวนการสำคัญที่ทำให้สามารถประมวลผลตัวอักษรลายมือเป็นตัวอักษรตัวพิมพ์ได้ถูกต้อง ดังนั้นการดึงเอาคุณลักษณะของตัวอักษร (Feature Extraction) เป็นขบวนการที่สำคัญ เพราะระบบการรู้จำขึ้นอยู่กับคุณสมบัติหลักๆสองประการ คือ

1. การเลือกคุณลักษณะเด่นของตัวอักษร (Feature)

2. การเลือก Recognizers ที่จะทำการรู้จำคุณลักษณะเด่นของตัวอักษรแต่ละตัว

ดังนั้นการหาและการเลือกคุณลักษณะเด่นของตัวอักษรจึงเป็นส่วนที่สำคัญเพราะคุณลักษณะเด่นจะบ่งบอกถึงตัวอักษรตัวนั้นๆและตัวอักษรแต่ละตัวก็จะมีคุณลักษณะเด่นเฉพาะตัวที่แตกต่างกัน

การหาคุณลักษณะเด่นของตัวอักษรนั้นจะหาโดยแบ่งตัวอักษรออกเป็นเซกเมนต์จุดแบ่งเซกเมนต์ของตัวอักษรนั้นจะเป็นจุดเปลี่ยนทิศทางในการเขียนตัวอักษรตามเข็มนาฬิกาหรือทวนเข็มนาฬิกา โดยพิจารณาทิศทางในการเขียนจากค่า Diffcode ซึ่งค่า Diffcode คือค่าความแตกต่างของรหัสลูกโซ่ GCC สองตัวที่อยู่ติดกันที่เชื่อมต่อกัน

$$\text{Diffcode}_i = \text{GCC}_{i+1} - \text{GCC}_i \quad (3)$$

จุดแบ่งเซกเมนต์จะเป็นจุดที่มีการเปลี่ยนแปลงของค่า Diffcode โดยจะมีการเปลี่ยนแปลงค่าจากค่าบวกที่อยู่ติดกันหลายๆค่าเป็นลบที่ติดอยู่กันหลายๆค่า หรือเปลี่ยนจากลบที่อยู่ติดกันหลายๆค่าเป็นบวกที่อยู่ติดกันหลายๆค่า การวาด

เส้นโค้งในทิศทางทวนเข็มนาฬิกาจะทำให้รหัสลูกโซ่ GCC มีค่าเพิ่มขึ้นทำให้ค่าความแตกต่าง Diffcode มีค่าเป็นบวก ในขณะที่การวาดเส้นโค้งไปในทิศทางเข็มนาฬิกาจะทำให้รหัสลูกโซ่ GCC มีค่าลดลงทำให้ค่าความแตกต่าง Diffcode นั้นมีค่าเป็นลบทั้งหมด

เมื่อได้จุดแบ่งเซกเมนต์ของตัวอักษรแล้วทำการหาค่าสะสมของ Diffcode ในแต่ละเซกเมนต์ โดยจะได้ค่าสะสมเป็นบวกหากเซกเมนต์นั้นเขียนไปในทิศทางทวนเข็มนาฬิกา และค่าสะสมของ Diffcode เป็นค่าลบหาก Segmnet นั้นเขียนไปในทิศทางเข็มนาฬิกา แต่เนื่องจากการหาในแบบนี้ยังมีบางตัวอักษรที่ต่างกันแต่เมื่อทำการหาเซกเมนต์แล้วได้จำนวนของเซกเมนต์เท่ากันและมีค่าสะสมความแตกต่าง Diffcode ที่ใกล้เคียงกัน ทำให้เราไม่สามารถแยกความแตกต่างของตัวอักษรดังกล่าวออกจากกันได้

Cumulativ Diffcode					
๕	1	2	3	4	5
seg 1	-0.584182	-0.612194	-0.637535	-0.596017	-0.612194
seg 2	0.47638	0.427905	0.427905	0.394501	0.427905
จ					
seg 1	-0.584182	-0.622869	-0.601073	-0.584182	-0.601073
seg 2	0.50133	0.584182	0.457577	0.414182	0.405177
ฅ					
seg 1	-0.552867	-0.584182	-0.616955	-0.584182	-0.622869
seg 2	0.47638	0.368578	0.33985	0.427905	0.448843
บ					
seg 1	-0.566144	-0.507643	-0.660669	-0.612194	-0.584182
seg 2	0.448843	0.465003	0.47638	0.427905	0.427905
ล					
seg 1	-0.552866	-0.627444	-0.517364	-0.50133	-0.552867
seg 2	0.457577	0.497357	0.47638	0.489854	0.517364

ตารางที่ 1 แสดงค่าสะสมความแตกต่าง Diffcode ที่ใกล้เคียงกันของตัวอักษรที่ต่างกัน ในการที่จะแยกตัวอักษรที่ต่างกันแต่มีจำนวนเซ็กเมนต์และค่าสะสมความแตกต่าง Diffcode ใกล้เคียงกันหรือเท่ากันออกจากกัน จะใช้ค่าความยาวในการลากตามแกน X และแกน Y จากซ้ายไปขวา ขวามาซ้าย จากบนลงล่าง และล่างขึ้นบน คิดเป็นเปอร์เซ็นต์ของความยาวในแต่ละเซ็กเมนต์ มาช่วยในการแยกตัวอักษรเหล่านี้ออกจากกัน

DiffLengthX เป็นความยาวระหว่างสองจุดที่อยู่ติดกันในแกน X

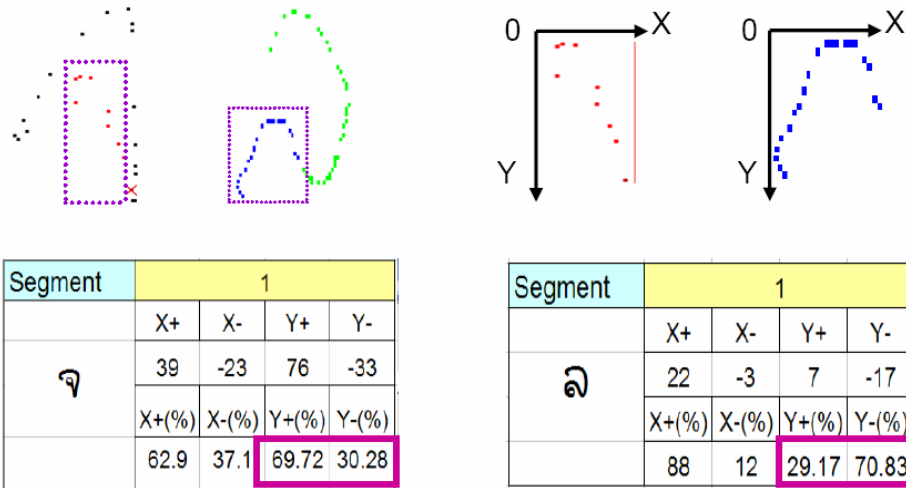
DiffLengthY เป็นความยาวระหว่างสองจุดที่อยู่ติดกันในแกน Y

$$\text{DiffLengthX } [i] = X [i+1] - X[i]$$

$$\text{DiffLengthY } [i] = Y [i+1] - Y[i]$$

(4)

ในแต่ละเซ็กเมนต์หา ผลรวมของ DiffLengthX ที่เป็นบวก, DiffLengthX ที่เป็นลบ, DiffLengthY และ DiffLengthY ที่เป็นลบแล้วคิดเป็นเปอร์เซ็นต์เทียบกับความยาวทั้งหมดของเซ็กเมนต์นั้น



รูปที่ 2.5 แสดงการเปรียบเทียบความแตกต่างของค่า X Y ของตัวอักษร จ และ ล ในเซ็กเมนต์

แรก

จ										
	Segment 1					Segment 2				
ตัวที่	sumdiff	X + (%)	X - (%)	Y + (%)	Y - (%)	sumdiff	X + (%)	X - (%)	Y + (%)	Y - (%)
1	-0.584182	70.4545	29.54545	83.63636	16.36364	0.50133	20.83333	79.16667	13.72549	86.27451
2	-0.622869	84.6154	15.38462	85.71429	14.28571	0.584182	13.46154	86.53846	18.86792	81.13208
3	-0.601073	81.25	18.75	85	15	0.457577	7.8125	92.1875	15.04425	84.95575
4	-0.584182	81.6667	18.33333	83.82353	16.17647	0.414182	22	78	32.06107	67.93893
5	-0.601073	63.4921	36.50794	79.54545	20.45455	0.405177	26.19048	73.80952	30.90909	69.09091
ล										
	Segment 1					Segment 2				
ตัวที่	sumdiff	X + (%)	X - (%)	Y + (%)	Y - (%)	sumdiff	X + (%)	X - (%)	Y + (%)	Y - (%)
1	-0.552866	71.0526	28.94737	40.90909	59.09091	0.457577	18.33333	81.66667	33.33333	66.66667
2	-0.627444	70.3704	29.62963	38.46154	61.53846	0.497357	28.57143	71.42857	27.27273	72.72727
3	-0.517364	67.6471	32.35294	37.5	62.5	0.47638	32.39437	67.60563	25	75
4	-0.50133	64.5161	35.48387	46.15385	53.84615	0.489854	32.43243	67.56757	26.47059	73.52941
5	-0.552867	71.4286	28.57143	35.29412	64.70588	0.517364	26.15385	73.84615	24.32432	75.67568

ตารางที่ 2 แสดงความแตกต่างของตัวอักษร จ และ ล ที่ใช้ความยาวในแกน X Y ในการแยก

การหาคุณลักษณะเด่นของตัวอักษรโดยใช้จุดเปลี่ยนทิศทางในการเขียนตัวอักษรเป็นจุดแบ่งเซ็กเมนต์ซึ่งพิจารณาจากค่า Diffcode ที่มีเครื่องหมายที่ต่างกัน และการที่ผลรวมของค่า Diffcode ไม่ขึ้นกับการหมุน ทำให้เส้นโค้งที่มีรูปร่างที่เหมือนกันแต่มีขนาดที่แตกต่าง แบ่งเซ็กเมนต์แล้วให้ผลที่เหมือนกันนั่นคือขนาดไม่มีผลต่อการแบ่งเซ็กเมนต์ถึงแม้ว่าการหาคุณลักษณะเด่นของตัวอักษรแบบนี้ทำให้ตัวอักษรที่ไม่เหมือนกันมีจำนวนเซ็กเมนต์ที่เท่ากัน และผลรวมค่าความแตกต่าง Diffcode ในเซ็กเมนต์ใกล้เคียงกันก็ตาม แต่ก็สามารถแยกอักษรดังกล่าวออกจากกันได้โดยการหาความยาวตามแกน X แกน Y และการหาความยาวของแต่ละ

ละเชิงเมนต์ ซึ่งทำให้การหาคุณลักษณะเด่นของตัวอักษรในลักษณะนี้สามารถแยกความแตกต่างของตัวอักษรได้ถูกต้องแม่นยำมากขึ้น และมีความยืดหยุ่นเพียงพอสำหรับตัวอักษรที่มีการหมุน หรือ มีการเปลี่ยนแปลงรูปร่างที่ไม่คงที่

2.2 ทฤษฎีที่ใช้ในการพัฒนาโครงงานนี้

2.2.1 ตัวอักษร แบ่งเป็นประเภทต่างๆดังนี้

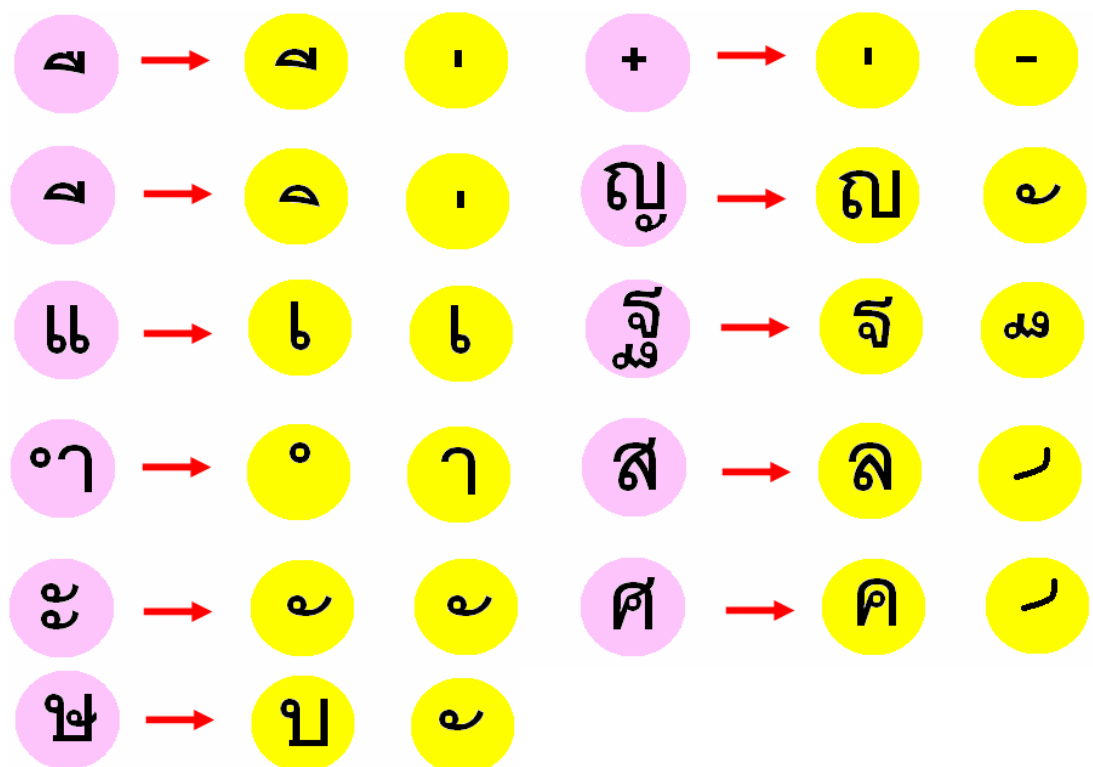
1. **อักษรโดด** คือ ตัวอักษรที่เขียนเสร็จได้ในครั้งเดียว

เช่น น.หนู ก.ไก่ สระอา สระอิ เป็นต้น

2. **อักษรที่อาจเกิดจากอักษรโดดมากกว่า 1 ตัว** ได้แก่

สระ อะ สระอิ สระอี สระแอะ สระอำ ไม้จัตวา ข.ฤๅษี ศ.ศาลา

ส.เสือ ญ.หญิง และ จ.ฐาน



3. **อักษรที่เกิดจากการเขียนอักษรโดดมากกว่า 2 ตัว**

ได้แก่ สระอี



2.2.2 การวิเคราะห์อักษรที่ไม่เป็นอักษรโคด

สามารถทำได้ 2 วิธีคือ

2.2.2.1 หาเอทพุทอักษรโคดก่อนตรวจการซ้อนทับ (Overlapping) ของลายมือเขียน เป็นวิธีที่ใช้ในระบบปัจจุบัน

ข้อดี

- สามารถเขียนตัวอักษร Single Stroke 2 ตัวทับกันได้

ข้อเสีย

- หากเอทพุทอักษรโคดที่ได้จากการรู้จำผิดพลาดก็จะทำให้เอทพุทที่แท้จริงผิดพลาดด้วย

2.2.2.2 ตรวจการซ้อนทับ (Overlapping) ก่อนประมวลผลหาอักษรเอทพุท

ข้อดี

- เอทพุทขึ้นกับผลการ Recognition น้อยกว่าวิธีการแรก
- จำนวน State ลดลงทำให้ความซับซ้อนของโปรแกรมลดลง

ข้อเสีย

- อาจเกิดความผิดพลาดในกรณีที่เขียนตัวอักษร Single Stroke 2 ตัวซ้อนทับกัน

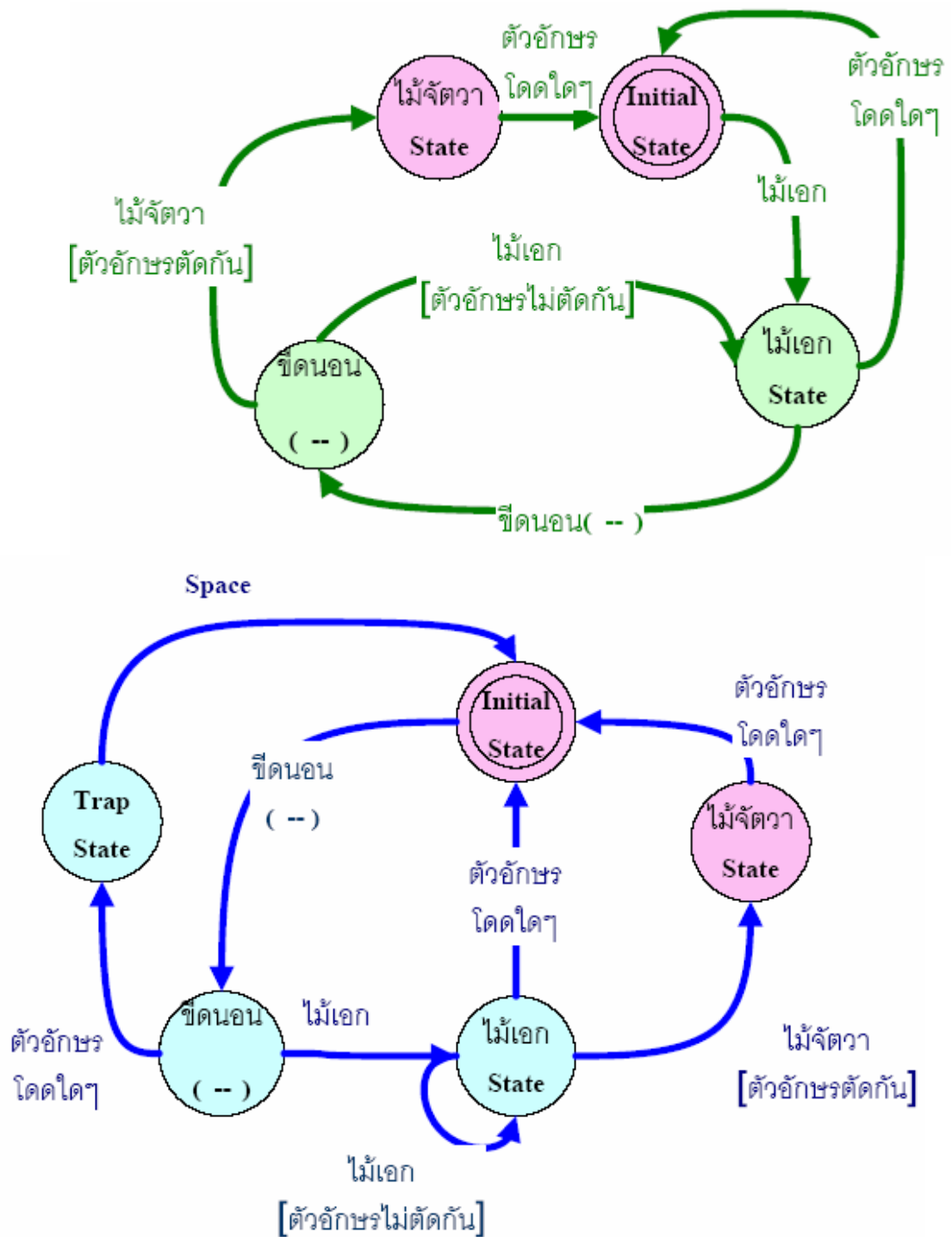
2.2.2.1 หาเอทพุทอักษรโคดก่อนตรวจการซ้อนทับ (Overlapping) ของลายมือเขียน เป็นวิธีที่ใช้ในระบบปัจจุบัน

ระบบวิเคราะห์อักษร Multiple Strokes ทำโดยเทียบอักษรโคดก่อนหน้า กับ อักษรโคดปัจจุบัน

การจัดการกับกรณีต่างๆข้างต้นโดยใช้ if else จะทำให้โปรแกรมเข้าใจยากและซับซ้อนเพราะต้องคำนึงถึงหลายกรณี

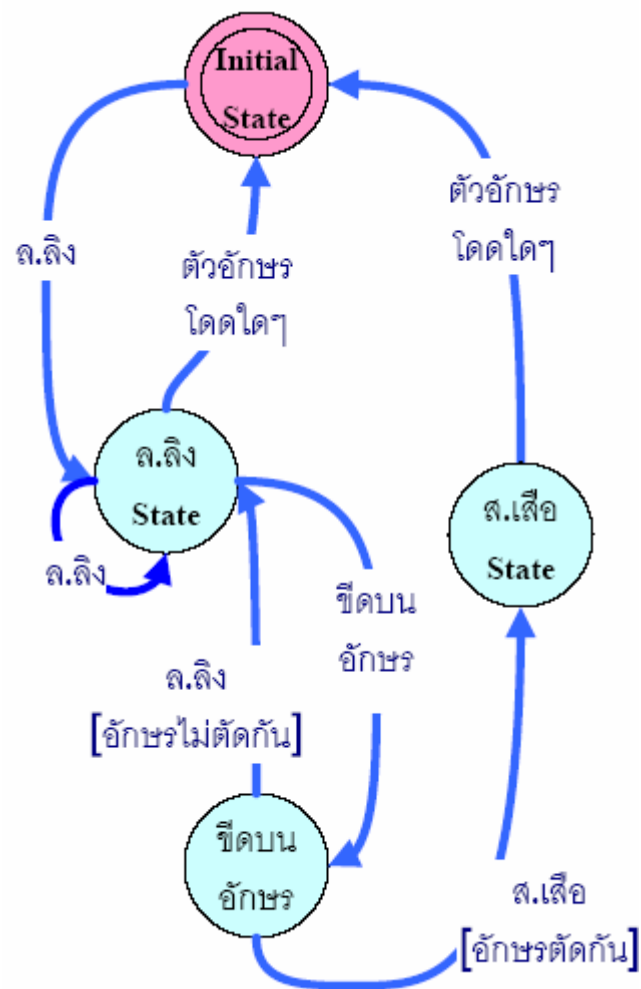
ด้วยเหตุนี้จึงนำ State Machine มาใช้ในการจัดการดังประกอบด้วย state และ transition ต่างๆ ดังนี้

1. State Machine ไม่จำกัด เมื่อรับอินพุตตัวอักษรเป็น อักษรโคดไม่เอก และ ชีคนอน ที่หากตัดกันจะได้ตัวอักษรไม่จำกัด



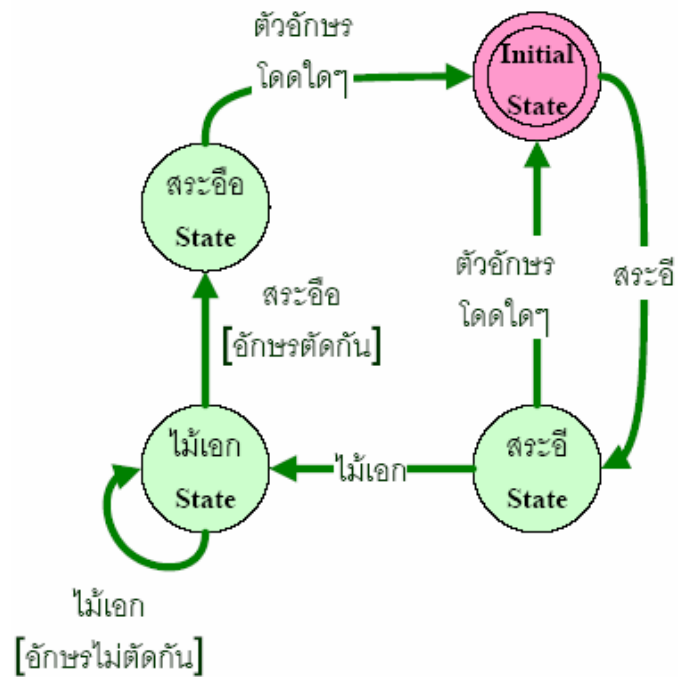
รูปที่ 2.6 แสดง State Machine ไม่จำกัด

4. State Machine ส.เสือ เมื่อรับอินพุตตัวอักษรเป็น อักษรโคดล.ถึงและ จีคบน
ตัวอักษร ที่หากตัดกันจะได้ตัวอักษรส.เสือ



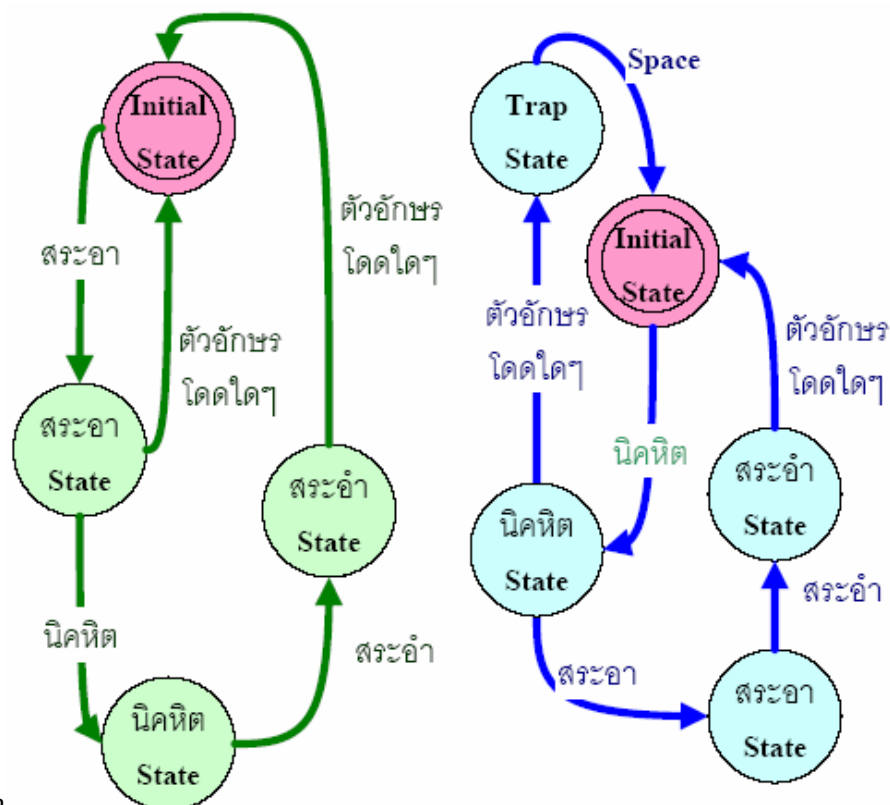
รูปที่ 2.9 แสดง State Machine ส.เสือ

5. State Machine สระอื้อ เมื่อรับอินพุตตัวอักษรเป็น อักษร โดค ไม่เอก และ สระอ
หรือ สระอซึ่งหากตัดกันจะได้สระอื้อ



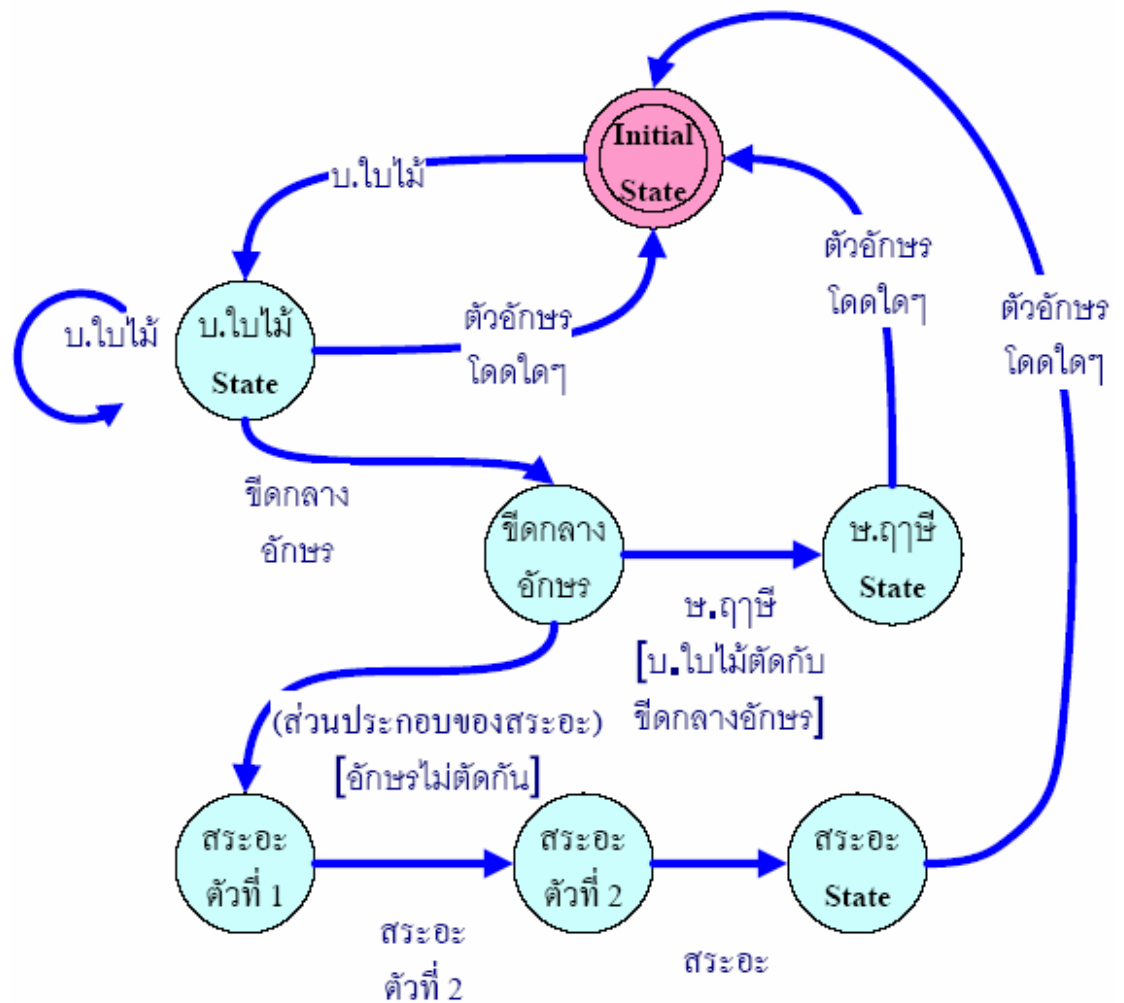
รูปที่ 2.10 แสดง State Machine สระอื้อ

6. State Machine สระอำ เมื่อรับอินพุตตัวอักษรเป็น อักษร โดค นิคฮิต และ สระอา
หรือ จะได้อักษรเอาท์พุทเป็นสระ



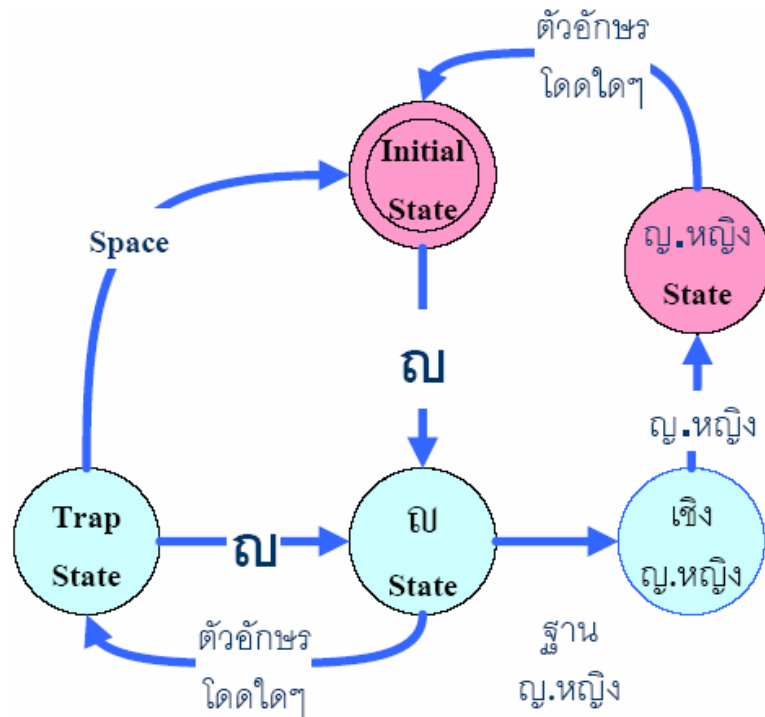
รูปที่ 2.11 แสดง State Machine สระอ้อ

7. State Machine ข.ฤๅษี เมื่อรับอินพุตตัวอักษรเป็น อักษรโดด บ.ไปไม้ และ จีดกลาง ตัวอักษร จะได้อักษรเอาท์พุทเป็น ข.ฤๅษี



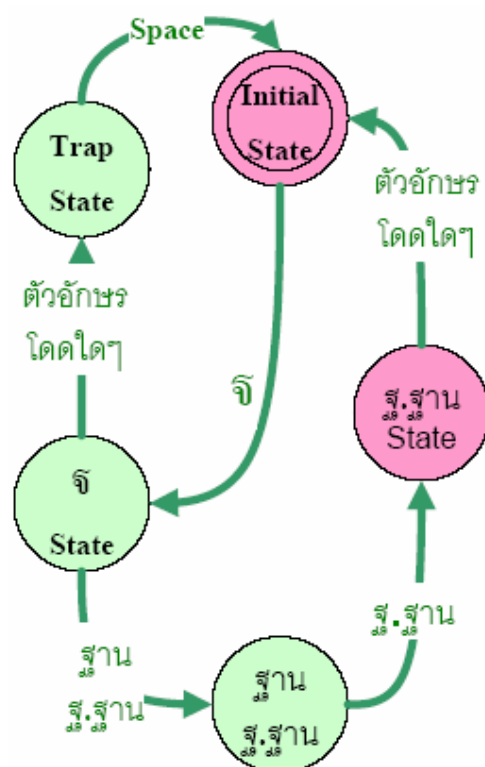
รูปที่ 2.12 แสดง State Machine ข.ฤๅษี

8. State Machine ญ.หญิง เมื่อรับอินพุตตัวอักษรเป็น อักษรโคด ญ.หญิง และ เซิง ญ.หญิง จะได้อักษรเอาพุทเป็น ญ.หญิง



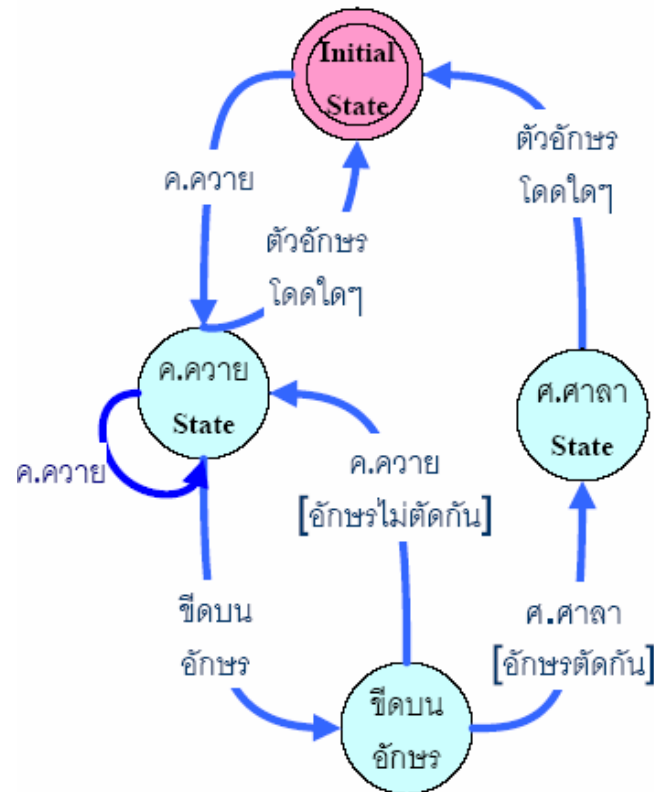
รูปที่ 2.13 แสดง State Machine ญ.หญิง

9. State Machine ฐ.ฐาน เมื่อรับอินพุตตัวอักษรเป็น อักษรโคด ฐ.ฐาน และ ฐาน ฐ.ฐาน จะได้อักษรเอาพุทเป็น ฐ.ฐาน



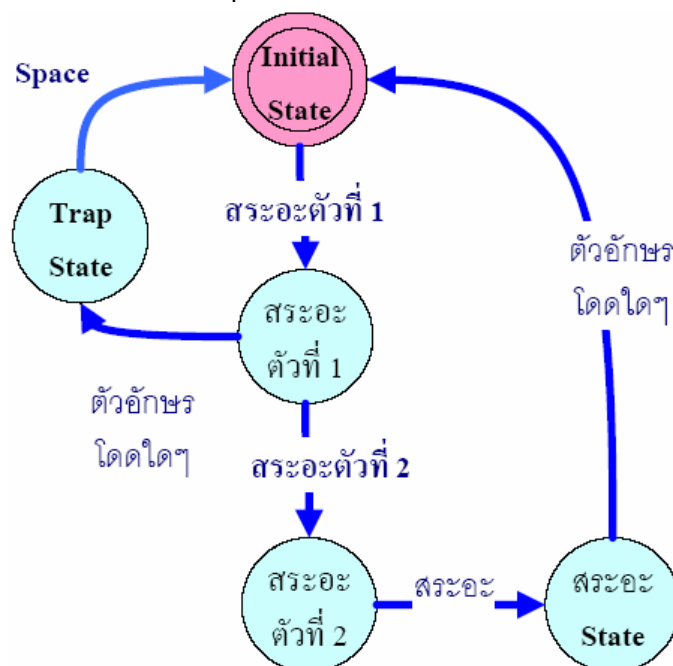
รูปที่ 2.14 แสดง State Machine ฐาน

10. State Machine ศ.ศาลา เมื่อรับอินพุตตัวอักษรเป็น อักษรโคด ค.ควาย และ จีคบน
ตัวอักษร จะได้อักษรเอาพุทเป็น ศ.ศาลา

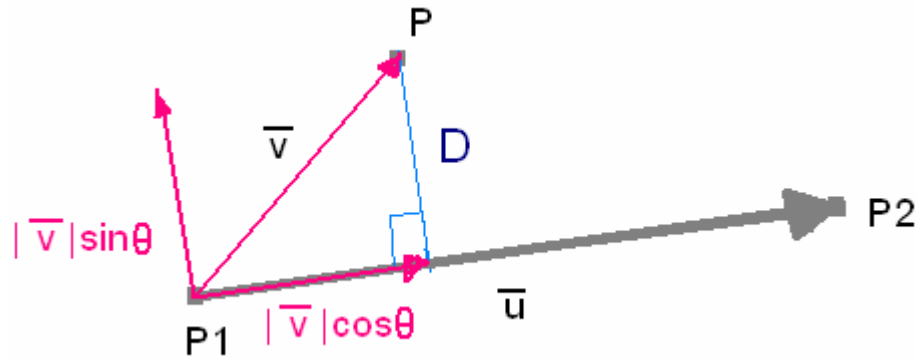


รูปที่ 2.15 แสดง State Machine ศ.ศาลา

11 State Machine สระอะ เมื่อรับอินพุตตัวอักษรเป็น อักษรโคด สระอะตัวที่1
และ สระอะตัวที่2 จะได้อักษรเอาพุทเป็น สระอะ



2.2.3.1. การหระยะห่างจากจุดใดๆไปยังเส้นตรง



รูปที่ 2.18 แสดงการหาระยะห่างจากจุด P ไปยังเส้นตรง P_1P_2

การหาระยะห่างจากจุด P ไปยังเส้นตรง P_1P_2

- สร้างเวกเตอร์ $\mathbf{u}$ ซึ่งเป็นเวกเตอร์จากจุด P_1 ไปยัง P_2
- สร้างเวกเตอร์ $\mathbf{v}$ ซึ่งเป็นเวกเตอร์จากจุด P_1 ไปยัง P
- ให้ระยะห่างจากจุด P ไปยังเส้นตรง P_1P_2 เป็น D

$$D = |\mathbf{v}| \sin \theta$$

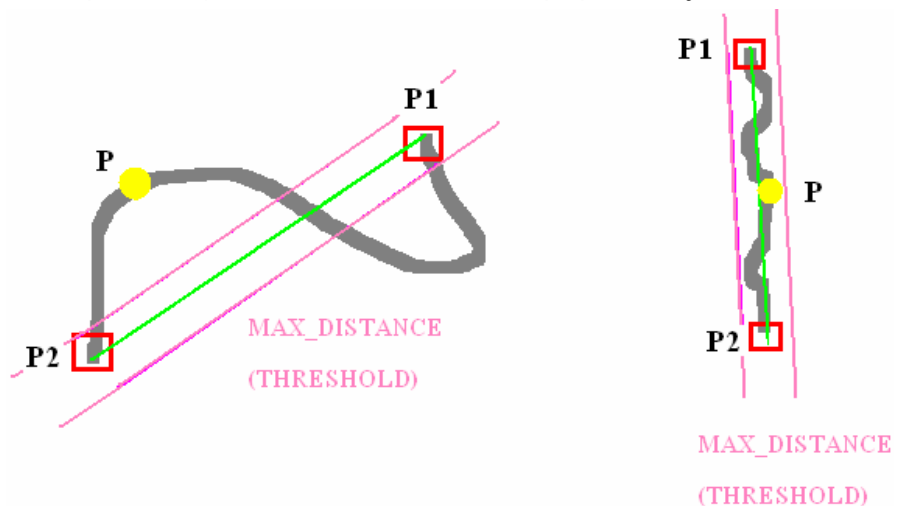
$$\text{และเนื่องจาก } |\mathbf{u} \times \mathbf{v}| = |\mathbf{u}| |\mathbf{v}| \sin \theta$$

ดังนั้น ระยะห่างจากจุด P ไปยังเส้นตรง P_1P_2 คำนวณได้จาก

$$D = \frac{|\mathbf{u} \times \mathbf{v}|}{|\mathbf{u}|}$$

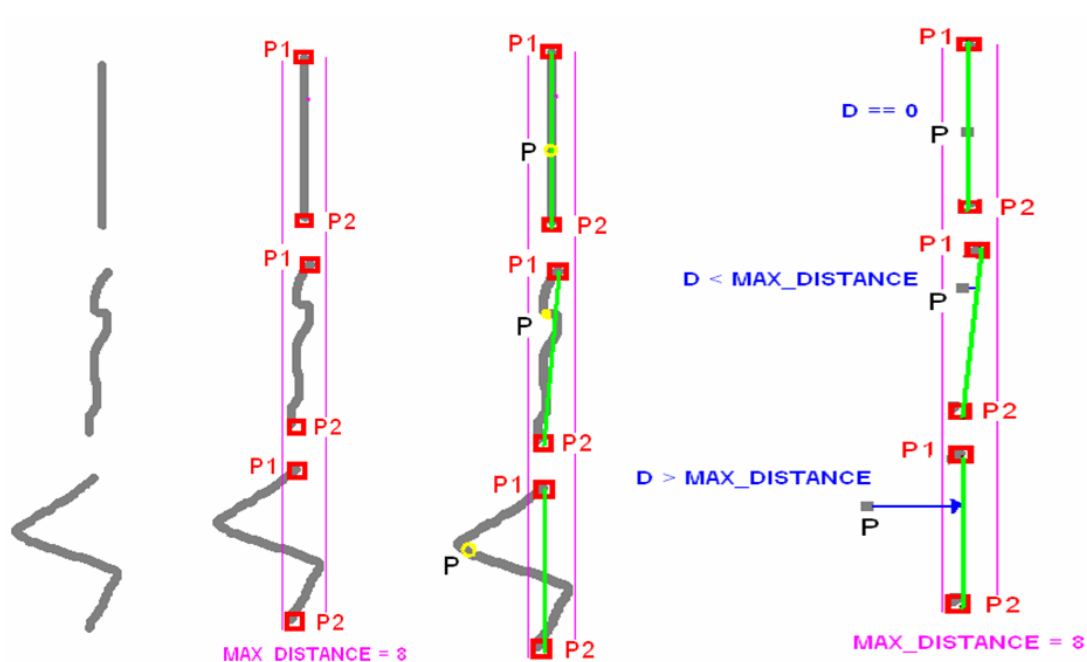
2.2.3.2. การวิเคราะห์ความเป็นเส้นตรงของเซตของพิกัด(x, y)

- ถ้ากำหนดให้จุด P_1 เป็นจุดเริ่มต้นของเส้น และ P_2 เป็นจุดสุดท้ายดังรูป



รูปที่ 2.19 แสดงการวิเคราะห์ความเป็นเส้นตรงของเส้นลายมือเขียน

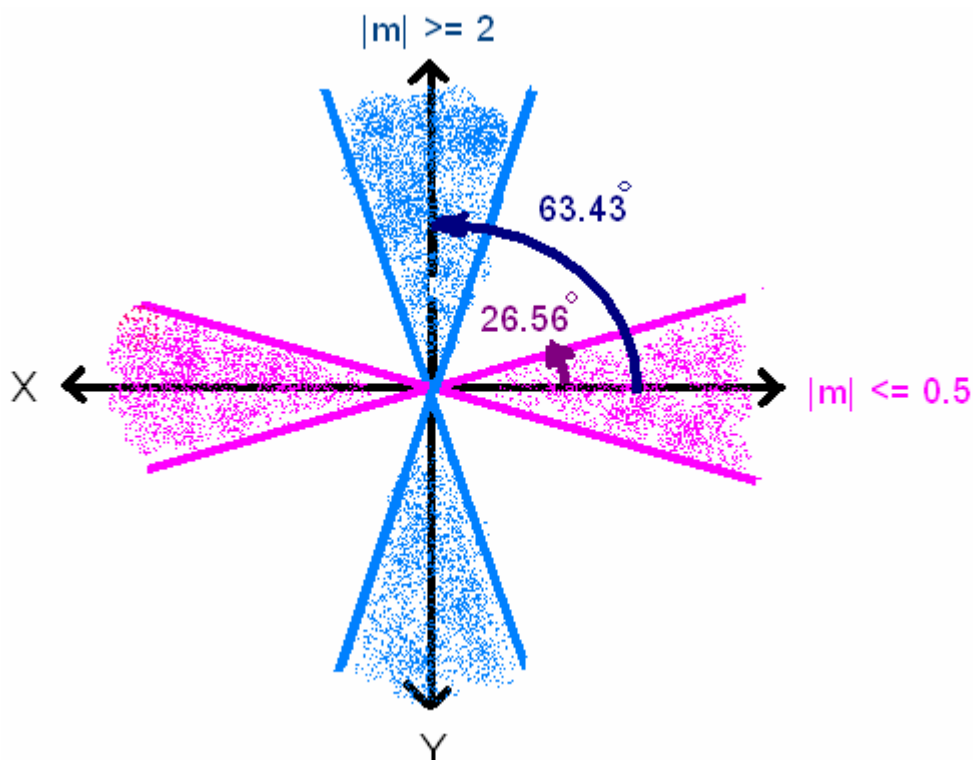
- b) สร้างเส้นตรงจาก P_1 ไปยัง P_2
- c) กำหนดให้จุดใดๆในเส้น เป็นจุด P
- d) หาระยะห่างจากจุด P ไปยังเส้นตรง P_1P_2
- e) หากมีจุด P ใดๆ ที่มีระยะห่างไปยังเส้นตรง มากกว่า ค่า $MAX_DISTANCE$ ซึ่งเป็นระยะห่างสูงสุดที่เป็นไปได้ที่กำหนดไว้ล่วงหน้าแล้ว (ค่า threshold) แสดงว่าเส้นดังกล่าวไม่เป็นเส้นตรง



รูปที่ 2.20 แสดงการวิเคราะห์ความเป็นเส้นตรงของเส้นลายมือเขียน

เมื่อผ่านการทดสอบนี้เซตของพิกัด (x, y) ที่ไม่เป็นเส้นตรงจะผ่านฟังก์ชันนี้ออกไปและเข้าสู่การวิเคราะห์หาอักษรตัวพิมพ์ด้วยวิธีปกติ (เรียกใช้โมดูลเดิมในการวิเคราะห์หาเอพ์พุท) แต่ถ้าหากเซตของพิกัด (x, y) เป็นเส้นตรงจะเข้าสู่การวิเคราะห์หาอักษรตัวพิมพ์ด้วยฟังก์ชันที่ใช้

2.2.3.3. ทฤษฎีตรีโกณมิติและความชันของเส้นตรง มาช่วยในการวิเคราะห์ดังนี้



รูปที่ 2.21 แสดงการวิเคราะห์รหัส ASCII ของเส้นลายมือเขียนที่เป็นเส้นตรง

a) หากเส้นตรงที่ได้มีความชันมากกว่าหรือเท่ากับสอง

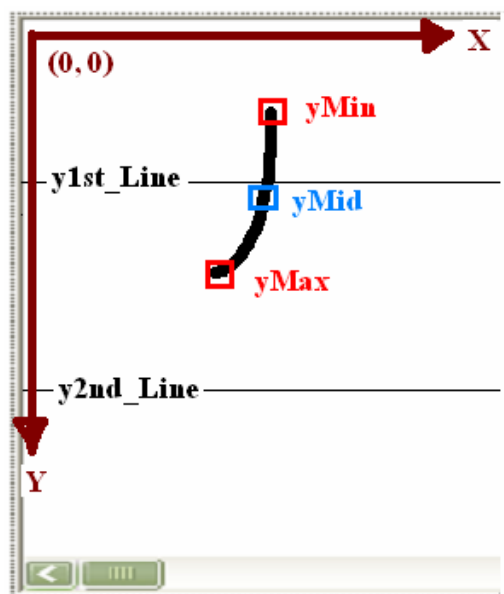
$|m| \geq 2$ แสดงว่า เส้นเอียงเป็นมุมประมาณ 63.43 องศา กับแนวแกน X
ประมาณให้เป็นเส้นตรงตามแนวแกน Y และให้เอาที่พูดเป็นไม้เอก

b) หากเส้นตรงที่ได้มีความชันน้อยกว่าหรือเท่ากับสอง

$|m| \leq 0.5$ แสดงว่า เส้นเอียงเป็นมุมประมาณ 26.56 องศา กับแนวแกน X
ประมาณให้เป็นเส้นตรงตามแนวแกน X และให้เอาที่พูดเป็นจิกกลางของ

ไม้จั่ว

2.2.4 การแยกบรรทัดตัวอักษร



รูปที่ 2.22 แสดงการแยกบรรทัดของลายมือเขียน

กำหนดให้จุดเริ่มต้นของเส้นบรรทัดแรก อยู่ที่ตำแหน่ง $(0, y1st_Line)$ และจุดเริ่มต้นของเส้นบรรทัดที่สอง อยู่ที่ตำแหน่ง $(0, y2nd_Line)$

- A. ค่า $yMax$ คือ ค่า y สูงสุดจากเส้นลายมือเขียน
- B. ค่า $yMin$ คือ ค่า y ต่ำสุดจากเส้นลายมือเขียน
- C. จะได้ค่า $yMid$ ซึ่งเป็น y ที่จุดกึ่งกลางเส้นลายมือเขียนมีค่า

$$yMid = (yMax + yMin) / 2$$

- D. นำค่า $yMid$ ที่ได้มาเปรียบเทียบกับเพื่อหาตำแหน่งของตัวอักษรดังนี้

- a) หาก $yMid < y1st_Line$ อักษรอยู่เหนือบรรทัด
- b) หาก $yMid > y2nd_Line$ อักษรอยู่ใต้บรรทัด
- c) $y1st_Line \leq yMid \leq y2nd_Line$ อักษรอยู่ในบรรทัด